

TÍTULO DEL PROYECTO

HYDRA GYM

NOMBRES DEL ESTUDIANTE

HENRIK ANDERSON OLOROSO GARCIA

MARCO DE TRABAJO SCRUM

DOCENTES INVOLUCRADOS

EDUIN SALAS
ANDRE NATARENO

CAMPUSLANDS
SALÓN QUETZAL I
RUTA NODE.JS
GUATEMALA, GUATEMALA
2026

1. SITUACIÓN PROBLEMA

A – Hydra Gym es un pequeño gimnasio con pocos clientes, a medida que el tiempo pasa, nuevos clientes se están volviendo cada vez más frecuentes, sin embargo ha surgido un problema: El gimnasio expandió sus instalaciones para nuevas máquinas y nuevos accesorios para el uso de sus clientes, sin embargo el gimnasio aún gestiona los datos de su clientes, los planes de entrenamiento, el seguimiento de sus progresos, la alimentación personalizada y sus pagos en hojas de excel simples; en un inicio al tener pocos clientes esta opción era viable aunque ya con ciertas complicaciones, sin embargo, ahora que el gimnasio está en crecimiento esta opción quedó indiscutiblemente obsoleta.

2. LEVANTAMIENTO DE REQUERIMIENTOS

El cliente solicita una aplicación de línea de comandos desarrollada con Node.js que permite gestionar integralmente la información de su gimnasio, reemplazando el uso de hojas de cálculo.

La aplicación deberá administrar clientes, planes de entrenamiento, contratos, seguimiento físico, nutrición, ingresos y egresos. Los datos se almacenarán en MySQL y el sistema deberá garantizar la consistencia de la información mediante transacciones reales.

3. REQUERIMIENTOS

3.1. Requerimientos Funcionales

- RF-01. Gestión de clientes

El sistema debe permitir:

1. Crear clientes.
2. Listar clientes.
3. Consultar la información de un cliente.
4. Actualizar los datos de un cliente.
5. Eliminar o desactivar clientes.
6. Asociar un cliente a uno o varios planes de entrenamiento.
7. Consultar los planes activos e históricos de un cliente.

- RF-02. Gestión de planes de entrenamiento

El sistema debe permitir:

1. Crear planes de entrenamiento.
2. Registrar el nombre del plan.
3. Registrar la duración.
4. Registrar las metas físicas.
5. Definir el nivel del plan:
 - Principiante.
 - Intermedio.
 - Avanzado.
6. Consultar los planes disponibles.
7. Asociar uno o varios clientes a un plan.

8. Renovar un plan asignado.
9. Cancelar un plan.
10. Finalizar un plan cuando se cumpla su duración.

- **RF-03. Gestión de contratos**

El sistema debe permitir:

1. Generar automáticamente un contrato al asignar un plan a un cliente.
2. Registrar las condiciones del contrato.
3. Registrar la duración.
4. Registrar el precio.
5. Registrar la fecha de inicio.
6. Registrar la fecha de finalización.
7. Asociar el contrato con el cliente correspondiente.
8. Asociar el contrato con el plan de entrenamiento correspondiente.
9. Consultar contratos activos, cancelados y finalizados.

- **RF-04. Seguimiento físico**

El sistema debe permitir:

1. Registrar avances físicos semanales.
2. Registrar el peso.
3. Registrar el porcentaje de grasa corporal.
4. Registrar medidas corporales.
5. Registrar fotografías.
6. Agregar comentarios sobre el progreso.
7. Consultar el progreso de forma cronológica.
8. Eliminar registros de seguimiento cuando corresponda.
9. Consultar el historial físico de un cliente.

- **RF-05. Gestión nutricional**

El sistema debe permitir:

1. Crear planes de alimentación.
2. Asociar un plan de alimentación con un cliente.
3. Asociar un plan de alimentación con un plan de entrenamiento.
4. Registrar alimentos por día.
5. Registrar las calorías estimadas de cada alimento.
6. Consultar los alimentos registrados.
7. Generar un reporte nutricional semanal.

- **RF-06. Gestión financiera**

El sistema debe permitir:

1. Registrar ingresos por mensualidades.
2. Registrar ingresos por sesiones individuales.
3. Registrar egresos por servicios.
4. Registrar egresos por suplementos.
5. Registrar otros gastos operativos.
6. Consultar ingresos por rango de fechas.
7. Consultar egresos por rango de fechas.
8. Consultar movimientos financieros asociados a un cliente.
9. Generar un balance financiero.

3.2. Requerimientos No Funcionales

- RNF-01: Calidad del código

El proyecto debe aplicar principios SOLID, Programación Orientada a Objetos y patrones de diseño para facilitar la comprensión y reutilización del código.

- RNF-02: Integridad de la base de datos

La base de datos no tiene que permitir que el usuario deje en blanco campos que son necesarios para el registro y el funcionamiento del negocio.

- RNF-03: La aplicación debe ofrecer mensajes claros

A medida que se usa la aplicación, ya sea que se esté ingresando un nuevo registro, debe mostrar mensajes exitosos o bien mensajes de error dado el caso que la acción no se haya podido realizar.

- RNF-04: La aplicación tiene que tener una interfaz intuitiva y fácil de usar

Debido a que es una aplicación de líneas de comandos, tiene que ser lo más intuitivo posible para que cuando se le presente al administrador del Gimnasio, aprenda a usarla en el menor tiempo posible.

- RNF-05: Organización modular

Para una mejor mantenibilidad y escalabilidad es necesario que el proyecto esté dividido y organizado en módulos.

3.3. Requerimientos Especiales

- **RE-01:** El contrato debe generarse de forma automática al asignar un plan
- **RE-02:** Si un cliente cancela un plan de entrenamiento debe hacer **rollback** de su seguimiento y contrato.
- **RE-03:** Los pagos se manejan con transacciones reales.
- **RE-04:** Se debe evidenciar en el código qué acciones son críticas y cómo asegura la consistencia de los datos.

3.4. Requerimientos técnicos

- RT-01: Uso de librerías npm relevantes
- RT-02: MySQL
- RT-03: Uso de transacciones en operaciones
- RT-04: Programación Orientada a Objetos
- RT-05: Aplicación de al menos 2 patrones de diseño
- RT-06: Aplicación de principios SOLID en clases y módulos

4. HISTORIAS DE USUARIO CON CRITERIOS DE ACEPTACIÓN

1. Gestión de Clientes

HISTORIA DE USUARIO			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-01	Actor	Administrador
NOMBRE DEL REQUERIMIENTO	Registro y administración de clientes		
Descripción			
Como administrador del gimnasio, quiero registrar, listar, consultar, actualizar y desactivar la información de los clientes para mantener un control centralizado y actualizado de sus datos personales.			
Funcionalidad			
El sistema permitirá crear un perfil de cliente con sus datos básicos (nombre, teléfono, correo, estado), consultar sus datos detallados en cualquier momento, actualizar información o cambiar su estado a inactivo si deja de asistir.			
Criterios de aceptación	1. El sistema valida que los campos obligatorios (nombre, contacto) no queden vacíos. 2. El sistema permite realizar búsquedas de clientes por ID o nombre. 3. Al desactivar un cliente, este no se elimina físicamente de la base de datos (borrado lógico) para preservar el historial financiero.		
Restricciones			
No se permite registrar dos clientes con el mismo documento de identidad/correo electrónico.			

2. Gestión de Planes de Entrenamiento

HISTORIA DE USUARIO			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-02	Actor	Administrador
NOMBRE DEL REQUERIMIENTO	Creación y configuración de planes de entrenamiento		
Descripción			
Como administrador, quiero crear y gestionar los planes de entrenamiento definidos por nivel (Principiante, Intermedio, Avanzado) para ofrecer opciones personalizadas a los clientes.			
Funcionalidad			
Permite registrar nuevos planes ingresando nombre, duración en días/meses, tarifa y metas físicas asociadas. Asimismo, permite consultar los planes disponibles y sus especificaciones.			

Criterios de aceptación	1. Cada plan debe tener asignado estrictamente un nivel validado: Principiante, Intermedio o Avanzado. 2. El sistema debe listar únicamente los planes activos para nuevas asignaciones. 3. El sistema calcula la fecha de finalización del plan a partir de la duración registrada.
Restricciones	
No se pueden modificar las condiciones de un plan si ya hay contratos activos asociados a él.	

3. Asignación de Plan, Generación Automática de contratos y pagos

HISTORIA DE USUARIO			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-03, RE-01, RE-03, RT-03	Actor	Administrador
NOMBRE DEL REQUERIMIENTO	Generación automática de contrato y registro transaccional de pago		
Descripción			
Como administrador, quiero asignar un plan de entrenamiento a un cliente para que el sistema genere automáticamente el contrato con sus fechas y registre el pago mediante una transacción segura en MySQL.			
Funcionalidad			
Al vincular un cliente a un plan, el sistema inicia una transacción en MySQL que crea el contrato en estado ACTIVO, registra el ingreso correspondiente en la gestión financiera y calcula la fecha de vencimiento.			
Criterios de aceptación	1. El contrato calcula automáticamente la fecha de inicio (fecha actual) y fecha de fin según la duración del plan. 2. La asignación del plan, la creación del contrato y el pago se ejecutan bajo una sola transacción ACID. 3. Si ocurre un fallo en el registro del pago, se cancela automáticamente toda la operación (ROLLBACK).		
Restricciones			
Un cliente no puede tener más de un contrato ACTIVO del mismo tipo simultáneamente.			

4. Gestión de planes de entrenamiento

HISTORIA DE USUARIO			
Prioridad: Media			
CÓDIGO DEL REQUERIMIENTO:	RF-03, RE-02	Actor	Administrador
NOMBRE DEL REQUERIMIENTO	Cancelación de contrato y revalidación de datos		
Descripción			
Como administrador, quiero cancelar un plan de entrenamiento activo a petición del cliente para revertir o desvincular de forma consistente sus registros asociados de seguimiento y contrato.			

Funcionalidad	
Al procesar una cancelación, el sistema actualiza el estado del contrato a CANCELADO y ejecuta un procedimiento controlado para deshabilitar o revertir los registros de seguimiento vinculados a dicha suscripción.	
Criterios de aceptación	1. El estado del contrato pasa a CANCELADO de manera inmediata. 2. El sistema confirma mediante una transacción la reversión o desvinculación limpia de los seguimientos del periodo. 3. El sistema genera un mensaje claro confirmando que la cancelación fue completada sin inconsistencias en MySQL
Restricciones	
No se pueden cancelar contratos cuya fecha de fin ya haya vencido (FINALIZADO).	

5. Seguimiento Físico del Cliente

HISTORIA DE USUARIO			
Prioridad: Media			
CÓDIGO DEL REQUERIMIENTO:	RF-04	Actor	Administrador / Entrenador
NOMBRE DEL REQUERIMIENTO	Registro y consulta del progreso corporal		
Descripción			
Como administrador, quiero registrar semanalmente los avances físicos del cliente (peso, % grasa, medidas, fotos) para consultar su evolución histórica ordenada cronológicamente.			
Funcionalidad			
Permite ingresar mediciones corporales periódicas asociadas a un cliente y desplegar reportes ordenados por fecha para evaluar el cumplimiento de sus metas físicas.			
Criterios de aceptación	1. El sistema registra peso, porcentaje de grasa, medidas y enlaces/rutas a fotos de progreso. 2. La consulta de avance se muestra en orden cronológico (del registro más reciente al más antiguo). 3. Permite eliminar o corregir un registro de seguimiento en caso de error de digitación.		
Restricciones			
Solo se pueden registrar avances a clientes con estado ACTIVO.			

6. Nutrición

HISTORIA DE USUARIO			
Prioridad: Media			
CÓDIGO DEL REQUERIMIENTO:	RF-05	Actor	Administrador / Nutricionista
NOMBRE DEL REQUERIMIENTO	Asignación de dietas y reportes nutricionales		
Descripción			
Como administrador, quiero asociar planes de alimentación a un cliente y su plan de entrenamiento para registrar sus comidas diarias y generar un reporte nutricional semanal.			

Funcionalidad	
Permite diseñar una pauta nutricional especificando alimentos por día con sus estimaciones calóricas y generar un resumen impreso en consola sobre el aporte nutricional del cliente.	
Criterios de aceptación	1. Permite vincular un plan nutricional tanto a un cliente como a un plan de entrenamiento específico. 2. Registra los alimentos desglosados por día de la semana y calorías estimadas. 3. Genera un reporte resumido con el total calórico diario/semanal.
Restricciones	
El total de calorías no puede ser un número negativo o mayor a un límite biológico predeterminado.	

7. Gestión financiera

HISTORIA DE USUARIO			
Prioridad: Alta			
CÓDIGO DEL REQUERIMIENTO:	RF-06	Actor	Administrador
NOMBRE DEL REQUERIMIENTO	Control de ingresos, egresos y balance general		
Descripción			
Como administrador, quiero registrar todos los movimientos financieros (mensualidades, venta de suplementos, servicios y egresos) para generar balances en rangos de fechas.			
Funcionalidad			
Módulo de contabilidad centralizado que registra ingresos y egresos, calcula balances (Ingresos - Egresos) y permite filtrar transacciones por cliente o intervalo de fechas.			
Criterios de aceptación	1. Clasifica las transacciones en categorías claras (Mensualidad, Sesión Individual, Suplementos, Servicios Operativos). 2. Permite consultar el balance financiero filtrando por fecha de inicio y fin. 3. Muestra el historial completo de pagos asociados a un cliente específico.		
Restricciones			
Toda transacción debe incluir una fecha, un monto mayor a cero y una descripción explícita.			

5. METODOLOGÍA Y MARCO DE TRABAJO SCRUM

Para el desarrollo del proyecto **Hydra Gym**, se adoptó una adaptación de la metodología ágil **Scrum para un único desarrollador (Solo Scrum)**, donde una sola persona asume la ejecución de los roles clave, la facilitación del proceso y la representación de los requerimientos del cliente.

5.1. Roles Asignados

- **Product Owner (Representante de Producto): Henrik Anderson Oloroso García**
 - *Responsabilidad:* Priorizar las historias de usuario del Backlog basándose en las necesidades operativas de Hydra Gym para sustituir el uso de hojas de cálculo.

- **Scrum Master (Facilitador del Proceso): Henrik Anderson Oloroso García**
 - *Responsabilidad:* Monitorear el avance del proyecto en ClickUp, organizar el cumplimiento del cronograma con fecha límite al **28 de septiembre de 2026** y resolver bloqueos técnicos.
- **Developer (Desarrollador): Henrik Anderson Oloroso García**
 - *Responsabilidad:* Diseñar el esquema relacional en MySQL, implementar la lógica en Node.js (POO, SOLID y transacciones ACID) y construir la interfaz CLI de consola.

5.2. Definición y Cronograma de Sprints (Deadline: 28 de Septiembre)

El proyecto se planificó y ejecutó en **3 Sprints** intensivos de 4 a 5 días cada uno, completando el ciclo completo de desarrollo antes del **28 de septiembre de 2026**.

Sprint 1: Base de Datos MySQL y Módulos Básicos

- **Fechas:** 14 de septiembre de 2026 – 18 de septiembre de 2026
- **Objetivo:** Diseñar el modelo relacional en MySQL, establecer la conexión con mysql2/promise en Node.js y desarrollar la gestión CRUD básica de Clientes (RF-01) y Planes de Entrenamiento (RF-02).
- **Entregables:**
 - Script de creación de tablas relacionales en MySQL.
 - Conexión por Pool transaccional configurada en Node.js.
 - Módulo funcional para la administración de clientes y planes.

Sprint 2: Transacciones ACID, Contratos, Cancelaciones y Finanzas

- **Fechas:** 19 de septiembre de 2026 – 23 de septiembre de 2026
- **Objetivo:** Implementar la lógica transaccional crítica en MySQL (BEGIN, COMMIT, ROLLBACK) para la generación automática de contratos (RF-03, RE-01), cancelaciones con limpieza de historial (RE-02) y movimientos financieros reales (RF-06, RE-03).
- **Entregables:**
 - Asignación automática de contratos al activar un plan.
 - Control transaccional seguro para pagos e ingresos/egresos.
 - Manejo consistente de cancelaciones mediante transacciones en MySQL.

Sprint 3: Seguimiento Físico, Nutrición, Patrones SOLID y CLI Interactivo

- **Fechas:** 24 de septiembre de 2026 – 28 de septiembre de 2026
- **Objetivo:** Concluir los módulos de seguimiento antropométrico (RF-04) y nutrición (RF-05), aplicar principios SOLID/Patrones de diseño (Repository, Singleton) y afinar la interfaz interactiva en consola (RNF-03, RNF-04).
- **Entregables:**
 - Registro cronológico de progreso físico y planes alimenticios.
 - Refactorización bajo principios SOLID y patrones de diseño (Repository, Singleton).
 - Interfaz de línea de comandos (CLI) terminada con menús interactivos, formato de tablas y validación de errores.

6. EVIDENCIA DE PLANTEAMIENTO DE PLATAFORMA DE TRABAJO

- Link del repositorio: <https://github.com/Anderson-Oloroso/hydra-gym.git>
- Link planificación: <https://app.clickup.com/90141677967/v/li/901421384546>

- Link de los videos de reuniones:
<https://drive.google.com/drive/folders/1JLchHYANYHezK2vpMPDIJ54TbuaoP4sY?usp=sharing>
- Link de evidencias:
<https://drive.google.com/drive/folders/1mnxieRSEbzy6KOFX1CCMnLEFsMm-RxOr?usp=sharing>
- Link de resultados y funcionamientos:
<https://drive.google.com/drive/folders/1mnxieRSEbzy6KOFX1CCMnLEFsMm-RxOr?usp=sharing>

7. CONCLUSIONES

Conclusiones generales del documento

1. La transición de hojas de cálculo simples en Excel a una solución de base de datos relacional MySQL optimizada con Node.js resuelve definitivamente los problemas de inconsistencia, pérdida de datos y escalabilidad en Hydra Gym.
2. La implementación de **transacciones ACID reales** (BEGIN, COMMIT, ROLLBACK) en MySQL garantiza que los procesos de cobro, asignación de contratos y cancelaciones se ejecuten con estricta integridad, evitando datos huérfanos o incongruencias financieras.
3. La estructuración del código bajo los principios **SOLID** y el uso de **patrones de diseño (Repository, Singleton)** permitieron construir un software mantenible, modular y preparado para escalar hacia una interfaz web en el futuro sin modificar la lógica de negocio.

Conclusiones de la reunión de retrospectiva del Sprint

1. **Qué funcionó bien:** La planificación en ClickUp dividida por Sprints facilitó el seguimiento riguroso de las tareas y la pronta identificación de componentes críticos de la base de datos.
2. **Qué se puede mejorar:** La complejidad inicial al manejar transacciones asíncronas con mysql2/promise en Node.js requirió un esfuerzo superior al estimado en la gestión de errores y *rollbacks*.
3. **Compromisos de mejora:** Implementar pruebas unitarias para los métodos de los repositorios para detectar fallos de datos antes de integrarlos con la interfaz de consola (CLI).